

4-话题管理与分布式架构实现.md

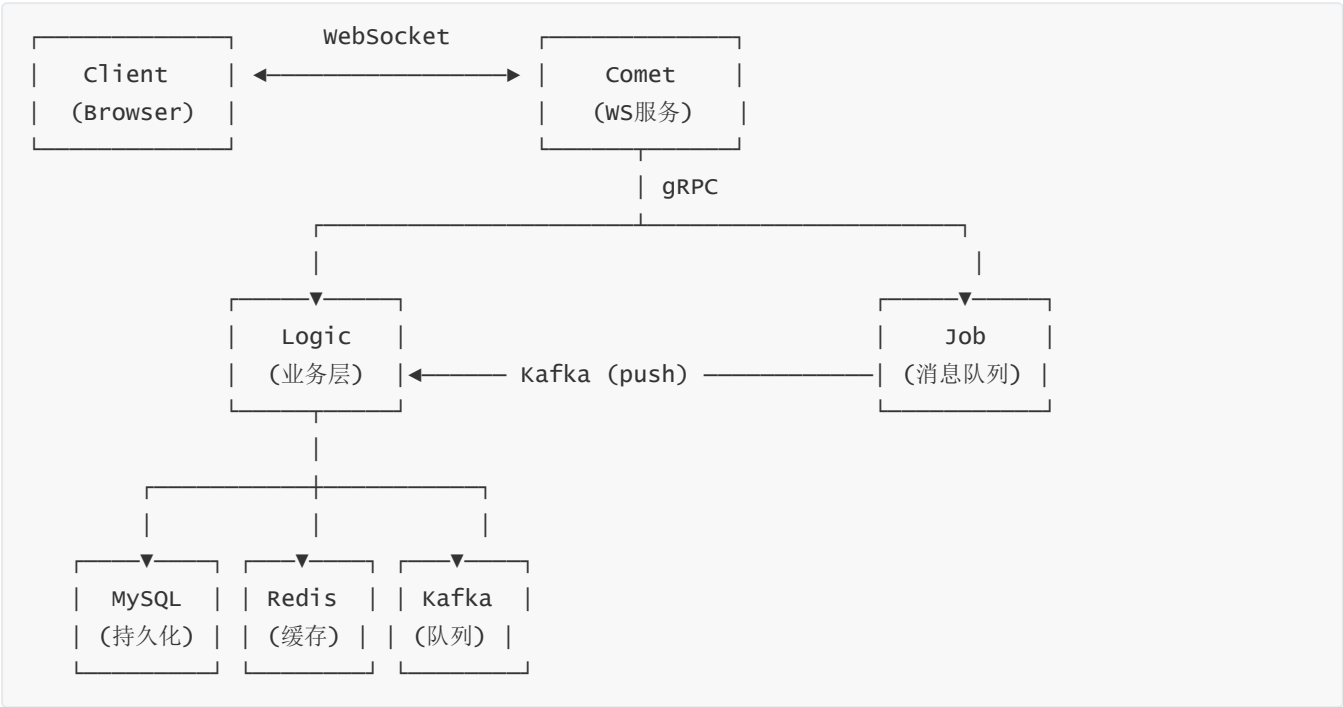
1. 概述

1.1 版本对比

04_room_subset 在 03_msg_subset 基础上新增了**房间/话题聊天室**功能，主要变化：

功能模块	03_msg_subset	04_room_subset
单聊消息	✅ 支持	✅ 支持
房间聊天	❌ 不支持	✅ 新增
消息类型	single_chat	single_chat + room
数据存储	MySQL(用户) + Redis	MySQL(用户+房间) + Redis(缓存)
扇出推送	点对点	点对点 + 房间扇出
会话管理	单聊会话	单聊 + 房间会话

1.2 架构概览



关键组件职责：

- **Comet:** WebSocket 长连接管理，消息上行/下行
- **Logic:** 业务逻辑处理，房间管理，消息路由

- **Job**: Kafka 消费者，按 comet_id 扇出推送
- **MySQL**: 用户、房间、成员关系持久化
- **Redis**: 路由、会话、未读、成员列表缓存

2. 新增功能列表

2.1 房间管理功能

功能	HTTP 接口	描述
创建房间	POST /api/room/create	管理员创建房间（仅创建，不自动加入其他用户）
房间列表	GET /api/room/list	获取所有房间信息（id/name/owner_id）
加入房间	POST /api/room/join	用户主动加入房间
退出房间	POST /api/room/leave	用户退出房间
房间成员	POST /api/room/users	查询房间成员列表

2.2 房间消息功能

功能	接口/协议	描述
HTTP 发送	POST /api/room/message/send	HTTP 方式发送房间消息
WS 发送	WebSocket 上行	target_type=room 消息
扇出推送	Job → Comet gRPC	按房间成员列表扇出推送
未读计数	Redis INCR	房间会话未读数管理

2.3 加入机制（无自动加入）

本版本**没有自动加入**能力：

- 用户不会因为“注册/登录”而被自动加入任何房间
- 房间创建也不会把“所有用户”自动加入该房间
- 加入房间只能通过用户显式调用 POST /api/room/join

3. 数据模型设计

3.1 MySQL 表结构

3.1.1 im_group（房间表）

```
CREATE TABLE IF NOT EXISTS `im_group` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(128) NOT NULL COMMENT '房间名称',  
  `owner_id` BIGINT NOT NULL COMMENT '创建者user_id',  
  `group_type` TINYINT NOT NULL DEFAULT 1 COMMENT '0=normal,1=chatroom,2=danmaku',  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `idx_owner` (`owner_id`),  
  UNIQUE KEY `uk_group_name` (`name`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

字段说明:

- `id`: 房间唯一标识
- `name`: 房间名称（唯一）
- `owner_id`: 创建者/管理员 user_id
- `group_type`: 房间类型（本版本统一为 1=chatroom）
- `created_at`: 创建时间

3.1.2 group_member（成员表）

```
CREATE TABLE IF NOT EXISTS `group_member` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `group_id` BIGINT NOT NULL,  
  `user_id` BIGINT NOT NULL,  
  `role` TINYINT NOT NULL DEFAULT 0 COMMENT '0=member,1=admin,2=owner',  
  `join_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_group_user` (`group_id`, `user_id`),  
  KEY `idx_user` (`user_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

字段说明:

- `group_id`: 房间 ID（外键关联 im_group）
- `user_id`: 用户 ID
- `role`: 成员角色（0=普通成员，1=管理员，2=群主）
- `join_at`: 加入时间

索引设计:

- `uk_group_user`: 保证一个用户在一个房间只有一条记录

- `idx_user`: 加速"查询用户加入的所有房间"

3.2 Proto 协议扩展

3.2.1 UpstreamMessageRequest (上行消息)

```
message UpstreamMessageRequest {  
    int64 from_user_id = 1;  
    string client_msg_id = 2;  
    string content_json = 3;  
    string comet_id = 4;  
    string conn_id = 5;  
    int64 received_time_ms = 6;  
    string msg_type = 7;  
    string target_type = 8;    // 新增: single_chat / room  
    int64 target_id = 9;      // 新增: room_id 或 对端 user_id  
}
```

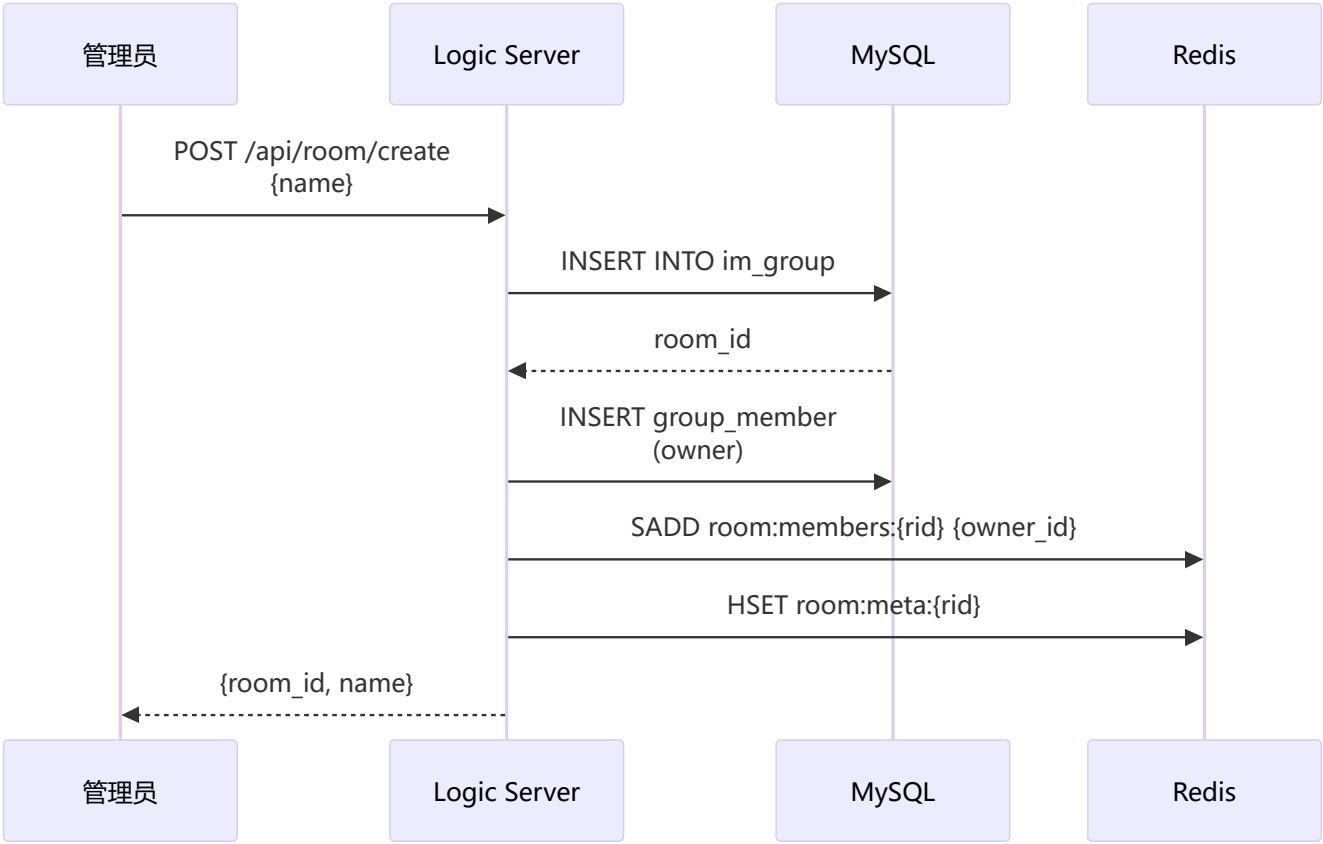
关键字段:

- `target_type`: 消息目标类型
 - `single_chat`: 单聊 (target_id 为对端 user_id)
 - `room`: 房间 (target_id 为 room_id)
- `target_id`: 根据 target_type 解释为不同含义

4. 核心功能设计

4.1 房间创建流程

4.1.1 时序图



4.1.2 核心代码

```
// room_store.cpp: CreateRoom
bool RoomStore::CreateRoom(const std::string& name, int64_t owner_id,
                           bool auto_join_all, int64_t* room_id,
                           std::string* err_msg) {
    // 说明：本版本不做任何“自动加入房间”，auto_join_all 参数仅保留兼容。
    (void)auto_join_all;
    // 1. 写 MySQL im_group 表
    int64_t rid = 0;
    if (!dao->CreateRoom(name, owner_id, 1, &rid, err_msg)) {
        return false;
    }

    // 2. Owner 自动加入 (role=2)
    dao->AddMember(rid, owner_id, 2, nullptr);

    // 3. 回读元信息并写 Redis 缓存
    RoomInfo info;
    if (dao->GetRoomById(rid, &info, nullptr)) {
        WarmRoomMetaCache(info); // HSET room:meta:{rid}
```

```

}

// 4. 写穿缓存: 仅 owner
redis_store->JoinRoom(owner_id, rid);
redis_store->ExpireRoomMembersCache(rid, members_cache_ttl_seconds);

if (room_id) *room_id = rid;
return true;
}

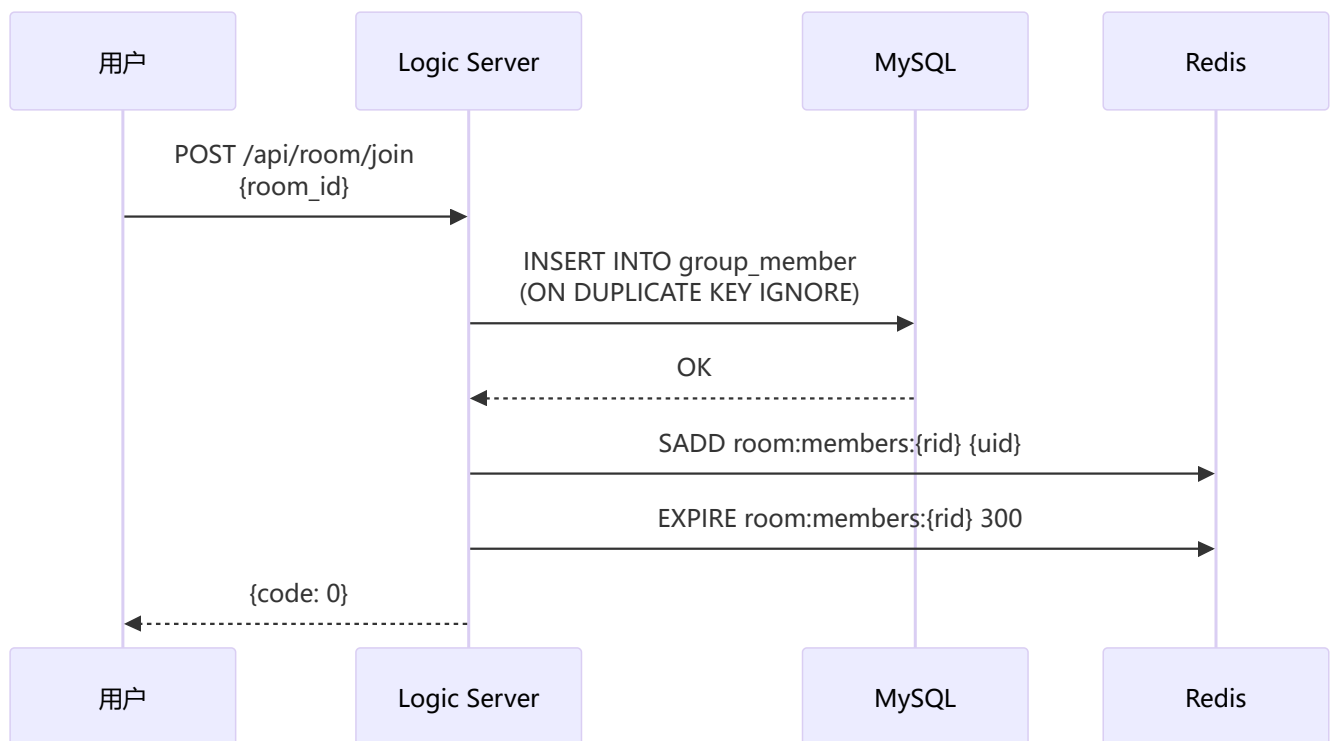
```

4.1.3 关键设计

1. **MySQL 持久化优先**: 房间元信息和成员关系必须落库
2. **缓存策略**:
 - 创建房间后写穿成员缓存 (只有 owner)
3. **事务性**: 当前未使用显式事务, 可接受短暂不一致

4.2 加入/退出房间流程

4.2.1 时序图



退出流程类似, 只是 `INSERT` 变为 `DELETE`, `SADD` 变为 `SREM`

4.2.2 核心代码

```

// room_store.cpp: JoinRoom
bool RoomStore::JoinRoom(int64_t user_id, int64_t room_id,
                          std::string* err_msg) {
    // 1. 写 MySQL (role=0 普通成员)

```

```
if (!dao_>AddMember(room_id, user_id, 0, err_msg)) {  
    return false;  
}  
  
// 2. 写穿 Redis 缓存  
if (redis_store_) {  
    redis_store_>JoinRoom(user_id, room_id); // SADD  
    redis_store_>ExpireRoomMembersCache(room_id,  
                                          members_cache_ttl_seconds_);  
}  
return true;  
}
```

4.2.3 Write-Through 缓存策略

写操作流程：

1. 先写 MySQL (Source of Truth)
2. 同步更新 Redis 缓存 (Write-Through)
3. 设置 TTL (默认 300s)

优点：

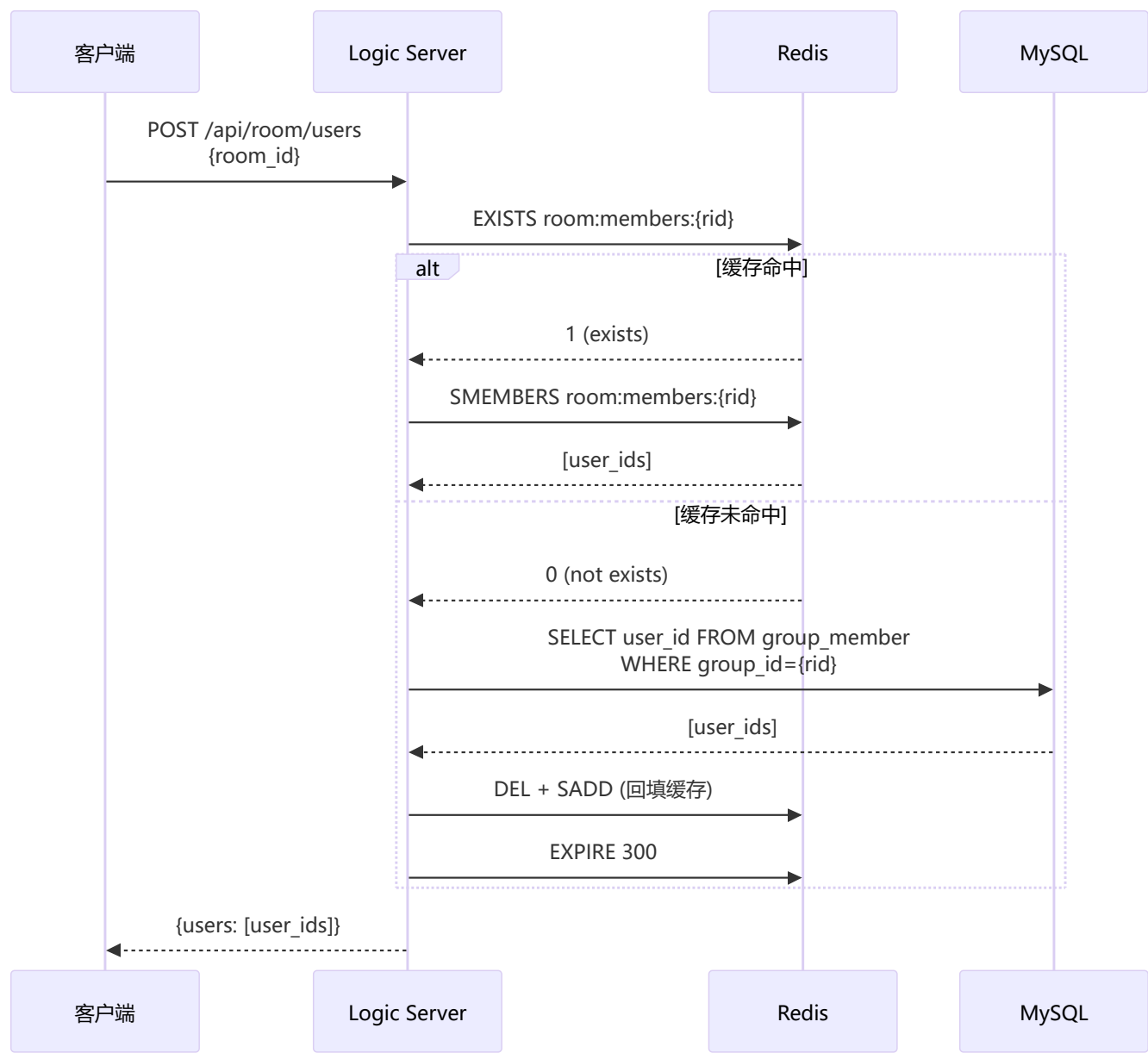
- 写入即可用，无需等待缓存失效
- 适合频繁加入/退出的场景

缺点：

- 若 Redis 失败，缓存与 DB 短暂不一致
- 本实现容忍此不一致 (Redis 仅作加速)

4.3 房间成员查询流程（Read-Through 缓存）

4.3.1 时序图



4.3.2 核心代码

```
// room_store.cpp: ListRoomMembers
bool RoomStore::ListRoomMembers(int64_t room_id,
                                std::vector<int64_t>* user_ids,
                                std::string* err_msg) {
    user_ids->clear();

    // 1. 检查缓存是否存在
    bool cache_exists = false;
    if (RoomMembersCacheExists(room_id, &cache_exists) && cache_exists) {
        // 2. 缓存命中，直接返回
        if (redis_store->ListRoomMembers(room_id, user_ids)) {
```

```

        return true;
    }
    user_ids->clear(); // Redis 异常, 回源
}

// 3. 缓存未命中, 从 MySQL 回源并回填
return WarmRoomMembersCacheFromDb(room_id, user_ids, err_msg);
}

bool RoomStore::WarmRoomMembersCacheFromDb(int64_t room_id,
                                             std::vector<int64_t>* members_out,
                                             std::string* err_msg) {

    // 从 MySQL 读取
    std::vector<int64_t> members;
    if (!dao_->ListMembers(room_id, &members, err_msg)) {
        return false;
    }

    // 回填 Redis (DEL + SADD + EXPIRE)
    if (redis_store_) {
        redis_store_->ReplaceRoomMembersCache(room_id, members,
                                                members_cache_ttl_seconds_);
    }

    if (members_out) *members_out = std::move(members);
    return true;
}

```

4.3.3 为什么需要 EXISTS 判断?

问题: Redis SET 无法区分"key不存在"和"空集合"

- SMEMBERS non_exist_key => []
- SMEMBERS empty_set_key => []

解决方案: 用 EXISTS 判断

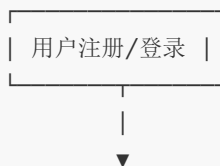
- EXISTS room:members:{rid} == 0 => 缓存未命中, 回源 MySQL
- EXISTS room:members:{rid} == 1 => 缓存命中 (可能是空集合)

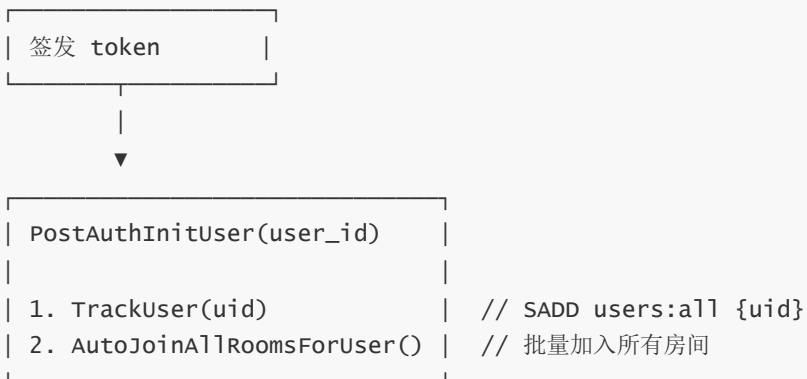
4.4 自动加入机制

4.4.1 新用户注册/登录后自动加入

触发时机: POST /api/register 或 POST /api/login 成功后

流程图:





核心代码:

```
// http_server.cpp: PostAuthInitUser
void HttpApiServer::PostAuthInitUser(int64_t user_id) {
    if (!redis_store_ || !room_store_) return;

    // 1. 记录到 users:all
    redis_store_>TrackUser(user_id);

    // 2. 自动加入所有现有房间
    room_store_>AutoJoinAllRoomsForUser(user_id, nullptr);
}

// room_dao.cpp: AutoJoinAllRoomsForUser
bool RoomDao::AutoJoinAllRoomsForUser(int64_t user_id,
                                       std::string* err_msg) {

    // 查询所有房间 ID
    std::vector<int64_t> room_ids;
    if (!ListRoomIds(&room_ids, err_msg)) return false;

    // 批量插入 (忽略重复)
    for (int64_t rid : room_ids) {
        AddMember(rid, user_id, 0, nullptr); // role=0
    }
    return true;
}
```

4.4.2 自动加入 (已禁用)

本版本按产品要求: **不做任何自动加入房间。**

- 创建房间: 只创建并默认仅创建者 (owner) 在房间里
- 注册/登录: 只完成注册/登录, 不会把用户自动加入任何既有房间
- 加入房间: 必须由用户显式调用 `POST /api/room/join`

说明: 为了兼容旧前端/旧文档字段, `/api/room/create` 仍可能看到/返回 `auto_join_all`, 但后端会忽略该参数并固定为 `false`。

4.4.3 通过“分享链接”加入房间 (Web Demo)

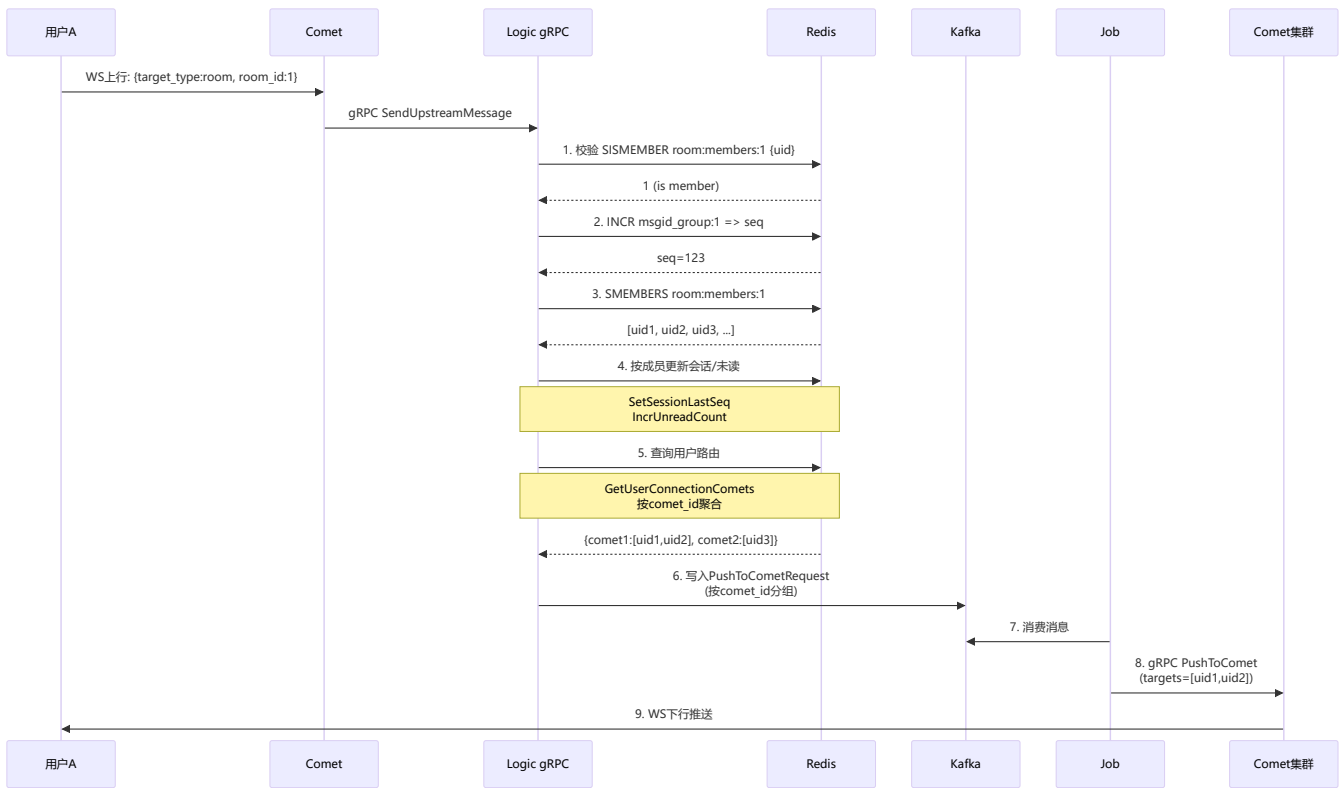
为方便测试，本版本在 Web Demo 的管理员页面提供“分享”按钮，一键生成可加入房间的链接：

- **链接格式：** `http://<host>:9010/room_chat.html?room_id=<rid>&join=1`
- **行为说明：**
 - 用户已登录：打开链接后会自动调用 `POST /api/room/join` 加入对应房间，并自动选中进入该房间
 - 用户未登录：页面会提示先登录；登录成功后会自动执行加入

注意：这里的“自动加入”是**用户通过分享链接显式触发**的体验优化，不是系统层面的默认自动加入机制。

4.5 房间消息发送流程（扇出推送）

4.5.1 整体时序图



4.5.2 关键步骤详解

步骤1：成员权限校验

```
// grpc_service.cpp: SendUpstreamMessage (target_type=room)
bool is_in = false;
if (!room_store->IsUserInRoom(from_user, room_id, &is_in, nullptr) || !is_in) {
    SetError(response->mutable_error(), 403, "user not in room");
    return ::grpc::Status::OK;
}
```

步骤2：分配房间级消息序号

```
// 房间维度严格递增的 msg_seq
int64_t msg_seq = 0;
std::string msg_id;
if (redis_store->NextGroupMsgId(room_id, &msg_seq)) {
    msg_id = "msgid_room:" + std::to_string(room_id) + "-" + std::to_string(msg_seq);
}
```

Redis 操作:

```
INCR msgid_group:<room_id> => 返回递增序号
```

步骤3: 拉取房间成员列表

```
std::vector<int64_t> members;
room_store->ListRoomMembers(room_id, &members, nullptr);
```

- 优先从 Redis `room:members:{rid}` 读取 (SMEMBERS)
- 若缓存未命中, 回源 MySQL 并回填缓存

步骤4: 更新会话与未读

```
// http_server.cpp: handleRoomSendMessage
std::string session_id = "r_" + std::to_string(room_id);
redis_store->SetSessionLastSeq(session_id, msg_seq);

for (int64_t uid : members) {
    redis_store->AddUserSession(uid, session_id);
    if (uid != from_user) {
        redis_store->IncrUnreadCount(uid, session_id, 1); // 发送者不增加未读
    }
    redis_store->SetUserSessionMeta(uid, session_id, msg_id, msg_seq,
                                    msg_type, now_ms, preview);
}
```

Redis 操作:

```
ZADD user:sessions:{uid} {timestamp} {session_id}
HINCRBY unread:{uid}:{session_id} count 1
HSET session:last_seq {session_id} {msg_seq}
HSET user:session_meta:{uid}:{session_id} ...
```

步骤5: 按 comet_id 聚合在线用户

```
std::unordered_map<std::string, std::vector<int64_t>> comet_to_users;
for (int64_t uid : members) {
    std::vector<std::string> comets;
    if (!redis_store_>GetUserConnectionComets(uid, &comets)) continue;
    for (const auto& cid : comets) {
        comet_to_users[cid].push_back(uid);
    }
}
```

Redis 操作:

```
SMEMBERS user:conns:{uid} => [{"comet_id}:{conn_id}", ...]
```

步骤6: 写入 Kafka

```
for (const auto& kv : comet_to_users) {
    const std::string& cid = kv.first;
    const auto& users = kv.second;

    PushToCometRequest preq;
    preq.set_comet_id(cid);
    *preq.mutable_message() = cm;
    for (int64_t uid : users) {
        auto* t = preq.add_targets();
        t->set_user_id(uid);
    }

    std::string payload;
    if (preq.SerializeToString(&payload)) {
        push_producer_>Send(cid, payload); // key=comet_id
    }
}
```

为什么按 comet_id 聚合?

优点:

1. 减少 Kafka 消息数: 100 个用户在同一 comet => 1 条消息
2. 减少 gRPC 调用: Job 批量推送给 comet
3. 降低网络开销: 序列化一次, 推送多个用户

示例:

- 房间 100 个成员
- 分布在 3 个 comet 节点
- 只需写 3 条 Kafka 消息 (而非 100 条)

4.5.3 HTTP 发送与 WebSocket 发送对比

维度	HTTP 发送	WebSocket 发送
接口	POST /api/room/message/send	WS 上行 JSON

维度	HTTP 发送	WebSocket 发送
鉴权	Authorization: Bearer <token>	WebSocket 握手时验证
适用场景	服务端、定时任务、压测	实时聊天、用户交互
返回值	同步返回 msg_id/msg_seq	ACK 消息异步返回
实现路径	Logic HTTP → Kafka → Job → Comet	Comet → Logic gRPC → Kafka → Job → Comet

核心差异：

- HTTP 方式直接在 Logic 层完成扇出逻辑
- WebSocket 方式需经过 Comet 转发到 Logic

4.6 房间会话与未读管理

4.6.1 会话 ID 设计

房间会话与单聊会话使用统一的会话管理机制，通过 session_id 前缀区分：

单聊会话：msgid:<小uid>:<大uid>-<seq>
房间会话：r_<room_id>

示例：

- 单聊：msgid:1:2-123 （用户1和用户2的会话，序号123）
- 房间：r_1 （房间ID=1的会话）

4.6.2 会话列表查询

用户调用 GET /api/session/list 可同时获取单聊和房间会话：

```
{
  "sessions": [
    {
      "session_id": "msgid:1:2",
      "last_msg_id": "msgid:1:2-123",
      "last_seq": 123,
      "msg_type": "text",
      "timestamp": 1702345678000,
      "preview": "Hello"
    },
    {
      "session_id": "r_1",
      "last_msg_id": "msgid_room:1-456",
      "last_seq": 456,
      "msg_type": "text",
      "timestamp": 1702345688000,
      "preview": "Room message"
    }
  ]
}
```

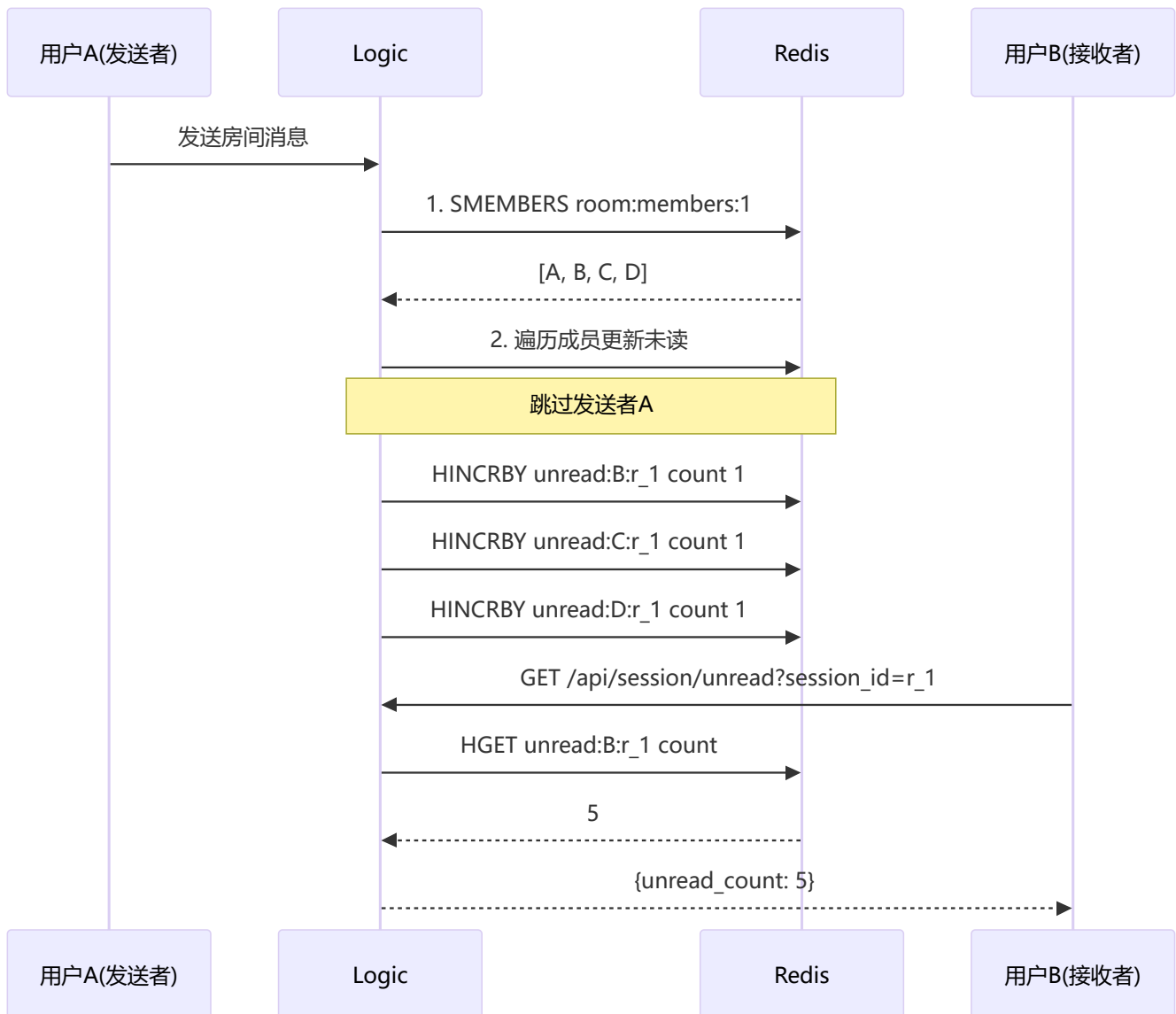
```
}
```

Redis 存储:

```
ZADD user:sessions:{uid} {timestamp} "msgid:1:2"  
ZADD user:sessions:{uid} {timestamp} "r_1"  
  
HGETALL user:session_meta:{uid}:msgid:1:2  
HGETALL user:session_meta:{uid}:r_1
```

4.6.3 房间消息未读流程

未读计数更新时机:



核心代码:

```
// http_server.cpp: handleRoomSendMessage
for (int64_t uid : members) {
    redis_store->AddUserSession(uid, session_id);
    if (uid != from_user) {
        // 发送者不增加自己的未读数
        redis_store->IncrUnreadCount(uid, session_id, 1);
    }
    redis_store->SetUserSessionMeta(uid, session_id, msg_id, msg_seq,
                                    msg_type, now_ms, preview);
}
```

已读标记:

```
# 用户B读取房间消息后
POST /api/session/mark_read
{
    "session_id": "r_1",
    "read_seq": 456
}

# Redis 操作
HSET unread:B:r_1 count 0
HSET unread:B:r_1 read_seq 456
```

4.6.4 未读数批量查询

```
POST /api/session/unread/batch
{
    "session_ids": ["msgid:1:2", "r_1", "r_2"]
}

# 响应
{
    "unread_map": {
        "msgid:1:2": {
            "unread_count": 3,
            "last_seq": 123,
            "read_seq": 120
        },
        "r_1": {
            "unread_count": 5,
            "last_seq": 456,
            "read_seq": 451
        }
    }
}
```

Redis 操作:

```
HGETALL unread:{uid}:msgid:1:2
HGETALL unread:{uid}:r_1
HGETALL unread:{uid}:r_2
```

4.7 单聊与群聊的区别

维度	单聊 (single_chat)	房间 (room)
session_id	msgid:<小uid>:<大uid>	r_<room_id>
msg_id	msgid:<小uid>:<大uid>-<seq>	msgid_room:<room_id>-<seq>
序号分配	INCR msgid:<小uid>:<大uid>	INCR msgid_group:<room_id>
推送目标	2 个用户（点对点）	N 个成员（扇出）
权限校验	无需校验（任意用户可互发）	必须是房间成员
会话创建	首次发消息自动创建	加入房间后创建
未读更新	对端 +1	所有成员（除发送者） +1

5. Redis 数据结构详解

5.1 房间相关 Key 设计

5.1.1 房间元信息缓存

Key: room:meta:<room_id>
Type: Hash
TTL: 600s（可配置）

字段:

- name: 房间名称
- owner_id: 创建者 user_id
- created_at_ms: 创建时间戳（毫秒）
- group_type: 房间类型（1=chatroom）

示例:

```
HGETALL room:meta:1
```

- 1) "name"
- 2) "技术讨论组"
- 3) "owner_id"
- 4) "1001"
- 5) "created_at_ms"
- 6) "1702345678000"
- 7) "group_type"
- 8) "1"

用途:

- 加速房间列表查询（避免频繁读 MySQL）
- 配合 `room_list_prefer_redis=true` 使用

5.1.2 房间成员列表缓存

Key: `room:members:<room_id>`
Type: Set
TTL: 300s（可配置）

成员: `user_id`（字符串形式）

示例:

`SMEMBERS room:members:1`

- 1) "1001"
- 2) "1002"
- 3) "1003"

操作:

- 加入: `SADD room:members:1 1004`
- 退出: `SREM room:members:1 1004`
- 判断: `SISMEMBER room:members:1 1004`
- 计数: `SCARD room:members:1`

缓存策略:

Read-Through + Write-Through

写操作（Join/Leave）:

1. 先写 MySQL（source of truth）
2. 同步更新 Redis（write-through）
3. 设置 TTL 300s

读操作（ListMembers/IsUserInRoom）:

1. `EXISTS room:members:{rid}`
2. 命中: 直接返回 `SMEMBERS`
3. 未命中: 回源 MySQL + 回填 Redis

5.1.3 用户维度的房间列表（预留）

Key: user:rooms:<user_id>

Type: Set

用途: 记录用户加入的所有房间 ID

示例:

SMEMBERS user:rooms:1001

1) "1"

2) "2"

3) "5"

注意: 本版本暂未使用此 key, 但预留用于未来优化"查询用户加入的房间列表"场景。

5.1.4 消息序号生成器

Key: msgid_group:<room_id>

Type: String (Integer)

用途: 房间维度的严格递增序号

示例:

INCR msgid_group:1 => 1

INCR msgid_group:1 => 2

INCR msgid_group:1 => 3

生成 msg_id:

"msgid_room:1-1"

"msgid_room:1-2"

"msgid_room:1-3"

5.2 会话与未读 Key 设计

5.2.1 用户会话列表

Key: user:sessions:<user_id>

Type: Sorted Set

Score: timestamp (毫秒)

成员: session_id

示例:

ZADD user:sessions:1001 1702345678000 "msgid:1001:1002"

ZADD user:sessions:1001 1702345688000 "r_1"

查询最近会话:

ZREVRANGE user:sessions:1001 0 19 WITHSCORES

5.2.2 会话元信息

Key: user:session_meta:<user_id>:<session_id>

Type: Hash

字段:

- last_msg_id: 最后一条消息 ID
- last_seq: 最后一条消息序号
- msg_type: 消息类型
- timestamp: 时间戳
- preview: 消息预览（前50字符）

示例:

HGETALL user:session_meta:1001:r_1

- 1) "last_msg_id"
- 2) "msgid_room:1-456"
- 3) "last_seq"
- 4) "456"
- 5) "msg_type"
- 6) "text"
- 7) "timestamp"
- 8) "1702345688000"
- 9) "preview"
- 10) "大家好，这是一条测试消息"

5.2.3 未读计数

Key: unread:<user_id>:<session_id>

Type: Hash

字段:

- count: 未读数（递增）
- read_seq: 已读到的序号
- last_seq: 最新消息序号

示例:

HGETALL unread:1001:r_1

- 1) "count"
- 2) "5"
- 3) "read_seq"
- 4) "451"
- 5) "last_seq"
- 6) "456"

操作:

- 新消息: HINCRBY unread:1001:r_1 count 1
- 已读: HSET unread:1001:r_1 count 0
HSET unread:1001:r_1 read_seq 456

5.2.4 会话最新序号

Key: session:last_seq

Type: Hash

字段: session_id => last_seq

示例:
HSET session:last_seq r_1 456
HSET session:last_seq msgid:1001:1002 123

用途:

- 快速判断会话是否有新消息
- 客户端断线重连后增量拉取

5.3 用户路由 Key 设计

Key: user:conns:<user_id>

Type: Set

成员: "<comet_id>:<conn_id>"

示例:
SMEMBERS user:conns:1001
1) "comet-1:conn-abc123"
2) "comet-2:conn-def456"

用途:

- 记录用户在哪些 comet 节点有活跃连接
- 消息推送时按 comet_id 聚合（减少 kafka 消息数）

生命周期:

- 用户上线: SADD user:conns:{uid} "{comet_id}:{conn_id}"
- 用户下线: SREM user:conns:{uid} "{comet_id}:{conn_id}"

5.4 完整 Redis Key 总览

用户相关:

└─ token:<token>

(String, TTL 24h) 用户鉴权

└─ user:conns:<user_id>

(Set) 用户在线连接

└─ user:sessions:<user_id>

(ZSet) 用户会话列表

└─ user:session_meta:<uid>:<session_id>

(Hash) 会话元信息

└─ unread:<user_id>:<session_id>

(Hash) 未读计数

房间相关:

└─ room:meta:<room_id>

(Hash, TTL 600s) 房间元信息缓存

└─ room:members:<room_id>

(Set, TTL 300s) 成员列表缓存

└─ msgid_group:<room_id>

(String) 消息序号生成器

会话相关:

└─ session:last_seq

(Hash) 会话最新序号

└─ msgid:<小uid>:<大uid>

(String) 单聊序号生成器

5.5 Redis 内存优化建议

对于生产环境，建议配置：

```
# 1. 启用 LRU 淘汰策略（缓存场景）
maxmemory-policy allkeys-lru
maxmemory 4gb

# 2. 为缓存 key 设置合理 TTL
room:meta:*      => 600s（10分钟）
room:members:*   => 300s（5分钟）
token:*          => 86400s（24小时）

# 3. 定期清理僵尸会话
user:sessions:*  => ZREMRANGEBYSCORE {uid} 0 {30天前}
```

6. 部署与测试

6.1 环境依赖

组件	版本要求	用途
MySQL	5.7+	用户、房间、成员持久化
Redis	5.0+	路由、缓存、会话、未读
Kafka	2.8+	消息队列
C++	17+	编译环境
CMake	3.15+	构建工具

6.2 编译步骤

```
cd /home/alientek/0voice/spark_push/04_room_subset
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

产物：

```
build/
├─ logic/logic_server
├─ comet/comet_server
├─ job/job_server
└─ web_demo/web_demo_server_04_room_subset
```

6.3 配置文件说明

6.3.1 logic.conf

```
# gRPC 监听端口
listen_port=9100

# HTTP API 端口
http_port=9101

# MySQL 配置
mysql_host=127.0.0.1
mysql_port=3306
mysql_user=root
mysql_password=your_password
mysql_database=spark_push

# Redis 配置
redis_host=127.0.0.1
redis_port=6379

# Kafka 配置
kafka_brokers=127.0.0.1:9092
kafka_push_topic=push_to_comet
```

6.3.2 comet.conf

```
# WebSocket 监听端口
listen_port=9000

# gRPC 服务端口（给 Job 调用）
comet_grpc_port=9105

# Comet 节点 ID（多节点部署时需唯一）
comet_id=comet-1

# Logic 地址
logic_server=127.0.0.1:9100
```

6.3.3 job.conf

```
# Kafka 配置
kafka_brokers=127.0.0.1:9092
kafka_topic=push_to_comet
kafka_group_id=job-group-1

# Comet 集群地址（逗号分隔）
comet_targets=comet-1:127.0.0.1:9105
```

6.4 启动顺序

```
# 1. 启动 Logic
./build/logic/logic_server -c conf/logic.conf

# 2. 启动 Comet
./build/comet/comet_server -c conf/comet.conf

# 3. 启动 Job
./build/job/job_server -c conf/job.conf

# 4. 启动 Web Demo (可选)
./build/web_demo/web_demo_server_04_room_subset -p 9010 -d ./build/web_demo/static
```

6.5 健康检查

```
# 检查 Logic HTTP
curl http://127.0.0.1:9101/api/room/list

# 检查 Comet WebSocket
wscat -c 'ws://127.0.0.1:9000/ws?token=test_token'

# 检查 Kafka 消费
kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 \
  --topic push_to_comet --from-beginning
```

6.6 完整测试流程

6.6.1 管理员创建房间

```
# 1. 管理员登录
ADMIN_TOKEN=$(curl -s -X POST 'http://127.0.0.1:9101/api/login' \
  -H 'Content-Type: application/json' \
  -d '{"account": "admin", "password": "0192023a7bbd73250516f069df18b500"}' \
  | jq -r '.token')

# 2. 创建房间
curl -X POST 'http://127.0.0.1:9101/api/room/create' \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"name": "技术讨论组", "auto_join_all": false}'
```

6.6.2 用户注册并加入房间

```
# 1. 注册用户
U1_TOKEN=$(curl -s -X POST 'http://127.0.0.1:9101/api/register' \
  -H 'Content-Type: application/json' \
  -d '{"account": "user1", "password": "e10adc3949ba59abbe56e057f20f883e", "name": "张三"}' \
  | jq -r '.token')
```

```
# 2. 加入房间
curl -X POST 'http://127.0.0.1:9101/api/room/join' \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $U1_TOKEN" \
  -d '{"room_id":1}'

# 3. 查看房间成员
curl -X POST 'http://127.0.0.1:9101/api/room/users' \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $U1_TOKEN" \
  -d '{"room_id":1}'
```

6.6.3 发送房间消息

方式1: HTTP 发送

```
curl -X POST 'http://127.0.0.1:9101/api/room/message/send' \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $U1_TOKEN" \
  -d '{"room_id":1,"msg_type":"text","content":"大家好!"}'
```

方式2: WebSocket 发送

```
# 连接 WebSocket
wscat -c "ws://127.0.0.1:9000/ws?token=$U1_TOKEN"

# 发送消息
{"msg_type":"text","target_type":"room","room_id":1,"content":"Hello WebSocket"}
```

6.6.4 查询未读数

```
# 获取会话列表
curl -H "Authorization: Bearer $U1_TOKEN" \
  'http://127.0.0.1:9101/api/session/list'

# 查询单个会话未读
curl -H "Authorization: Bearer $U1_TOKEN" \
  'http://127.0.0.1:9101/api/session/unread?session_id=r_1'

# 标记已读
curl -X POST 'http://127.0.0.1:9101/api/session/mark_read' \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $U1_TOKEN" \
  -d '{"session_id":"r_1","read_seq":10}'
```

6.7 Web Demo 测试

6.7.1 访问地址

首页: `http://127.0.0.1:9010/`
用户聊天室: `http://127.0.0.1:9010/room_chat.html`
管理员面板: `http://127.0.0.1:9010/admin_room.html`

6.7.2 测试场景

场景1: 管理员创建房间

1. 访问 `admin_room.html`
2. 使用 `admin / admin123` 登录
3. 点击"创建房间", 输入房间名称
4. 查看房间列表, 确认创建成功

场景2: 用户加入并聊天

1. 打开两个浏览器窗口访问 `room_chat.html`
2. 分别注册/登录不同用户 (`user1`, `user2`)
3. 点击"刷新房间列表"
4. 点击房间名加入
5. 在一个窗口发送消息, 观察另一窗口是否实时接收

场景3: 未读消息

1. 用户A发送消息后关闭页面
2. 用户B接收消息但不点开
3. 刷新页面, 观察会话列表是否显示未读数

6.8 常见问题排查

6.8.1 房间消息发送失败

错误: `403 user not in room, join first`

原因: 用户未加入房间

解决:

```
curl -X POST 'http://127.0.0.1:9101/api/room/join' \  
-H "Authorization: Bearer $TOKEN" \  
-d '{"room_id":1}'
```

6.8.2 消息发送成功但收不到推送

排查步骤:

1. 检查 Job 是否启动并消费 Kafka

```
# 查看 Job 日志  
tail -f /path/to/job.log | grep "consume message"
```

2. 检查 Kafka topic 是否有消息

```
kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 \  
--topic push_to_comet --from-beginning
```

3. 检查用户路由是否正确

```
redis-cli  
> SMEMBERS user:conns:1001
```

4. 检查 Comet gRPC 端口配置

```
# job.conf 的 comet_targets 必须与 comet.conf 的 comet_grpc_port 一致  
comet_targets=comet-1:127.0.0.1:9105
```

6.8.3 缓存不一致

现象: 用户加入房间后, 查询成员列表没有自己

原因: Redis 缓存未更新或 TTL 过期

解决:

```
redis-cli  
> DEL room:members:1 # 清除缓存, 下次查询会回源 MySQL
```

6.8.4 MySQL 连接失败

错误: no mysql connection

排查:

1. 检查 MySQL 服务是否启动

```
systemctl status mysql
```

2. 检查数据库和表是否创建

```
mysql -uroot -p  
> USE spark_push;  
> SHOW TABLES;
```

3. 检查配置文件密码是否正确

```
cat conf/logic.conf | grep mysql_password
```

7. 性能优化建议

7.1 Redis 缓存优化

```
// 1. 批量查询房间成员（减少 RTT）
使用 Redis Pipeline:
PIPELINE
  SMEMBERS room:members:1
  SMEMBERS room:members:2
  SMEMBERS room:members:3
EXEC

// 2. 使用 Lua 脚本原子操作
-- 加入房间并更新 TTL
redis.call('SADD', KEYS[1], ARGV[1])
redis.call('EXPIRE', KEYS[1], ARGV[2])

// 3. 未读数批量更新
PIPELINE
  HINCRBY unread:1001:r_1 count 1
  HINCRBY unread:1002:r_1 count 1
  HINCRBY unread:1003:r_1 count 1
EXEC
```

7.2 MySQL 优化

```
-- 1. 为高频查询添加索引
CREATE INDEX idx_group_id ON group_member(group_id);
CREATE INDEX idx_user_id ON group_member(user_id);

-- 2. 批量插入成员（自动加入场景）
INSERT INTO group_member (group_id, user_id, role)
VALUES (1, 1001, 0), (1, 1002, 0), (1, 1003, 0)
ON DUPLICATE KEY UPDATE role=role;

-- 3. 使用 COUNT(*) 优化成员统计
SELECT COUNT(*) FROM group_member WHERE group_id = 1;
```

7.3 Kafka 优化

1. 增加分区数（提高并行度）

```
kafka-topics.sh --alter --topic push_to_comet --partitions 10
```

2. 配置合理的批次大小

```
batch.size=16384
```

```
linger.ms=10
```

3. 启用压缩

```
compression.type=snappy
```